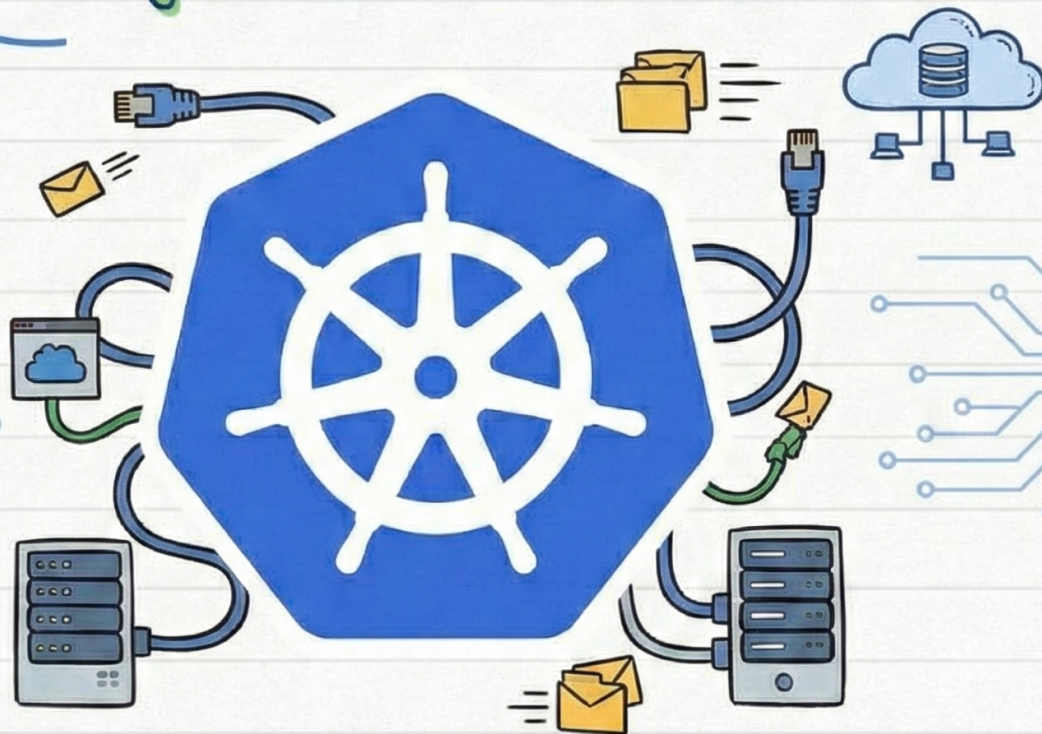


Kubernetes Simplified – Part 2



Handwritten By:
Vaibhav Matkar

⚡ Service → stable network endpoint used for communication.

⚡ service components

① clusterIP – default servicetype → make service reachability within cluster only. i.e exposes service to internal IP's [communication within cluster]

```
apiVersion: v1
kind: Service
metadata:
  labels:
    k8s-app: kube-dns
    kubernetes.io/cluster-service: "true"
    kubernetes.io/name: CoreDNS
  name: kube-dns
  namespace: kube-system
spec:
  clusterIP: 10.96.0.10
  ports:
    - name: dns
      port: 53
      protocol: UDP
      targetPort: 53
    - name: dns-tcp
      port: 53
      protocol: TCP
      targetPort: 53
  selector:
    k8s-app: kube-dns
  type: ClusterIP
```


② **Nodeport** – Expose service outside the cluster by opening port on every node. [communication outside cluster]

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - port: 80
      # By default and for convenience, the `targetPort` is set to
      # the same value as the `port` field.
      targetPort: 80
      # Optional field
      # By default and for convenience, the Kubernetes control plane
      # will allocate a port from a range (default: 30000-32767)
      nodePort: 30007
```

③ **Externalname** → just maps service name to external DNS

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

④ **loadbalancer** → created by cloud provider
exposes service to internet with public ip / DNS

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  clusterIP: 10.0.171.239
  type: LoadBalancer
status:
  loadBalancer:
    ingress:
      - ip: 192.0.2.127
```

Network communication in kubernetes

- ① container to container → via local host as belongs to same network namespace
- ② pod to pod within same Node → Each pod has its own ip so communicate via IP's using CNI
- ③ pod to pod across different Nodes (same cluster)
→ CNI plugin setup routing/overlay network across nodes to communicate

Kubernetes Network Security & Access control

- ① Network policies → rules that restricts traffic inside cluster
- ② RBAC → granular permissions for users & services
- ③ CNI → plugin based interface allows 3rd party network providers to give additional network security

Ingress in kubernetes

↳ App layer (7th)

↓
mgmt of incoming traffic to cluster
using adv routing & rules for HTTP/HTTPS

Functions

① Load Balancing

↓
distribute traffic to
different instances
of service.

② Routing

↓
distribute traffic
to multiple services
correctly using
routing rules

③ Access
control

↓
prevents un-
autho-access
to any service
based on ip.

④ No need of
TLS certifi-
cation for
every service
instead use
in Ingress

Encr → Decry

Working of Nodeport service

↓
Allows to expose container/application to outside world

i> create Nodeport service in k8 manifest file

mention  Target port → container listening
port to be exposed to outside world
ie. Nodeport

↓
manually / auto [30,000 - 32767]

ii> kubernetes maps Target port to Nodeport

iii> to access application outside kubernetes cluster → http://node-ip/nodeport

kube proxy

• kube proxy → Agent installed  as daemonset
on every node  Linux process

↓
middleware between service & pod

• function → monitor changes in service and endpoints [pod ip's] → create network rules
↓
decides which request will go to which pod

→ pods are ephemeral → service provides

Load
Balancing
between pods

Stable ip
to pods

DNS name
to pods

Working of kube proxy

- ① Any new service / endpoints added → API server inform this to kube proxy
- ② kube proxy create NAT rule for this changes.
i.e. map service ip to pod ip → when request come to service it is redirected to pod

NAT [Network Address translation]

Rule that forwards ip address/port (1st)
to ip address/port (2nd)

SNAT

Internal
private
IP → public
IP

DNAT

redirect external
request to appropriate
destination

kube proxy modes

User space mode

- Use iptables

↓
• Traffic
↓
iptables ← redirect rule
[@kernel space]
↓
kube-proxy ← contains service to
local port [@user space] pod rule

↓
forwarded respective pods

Latency & Unpopular

iptables mode

- uses iptables

- Traffic

↓
iptables contains service to pod mapping rule added by kube-proxy

↓
forwarded respective pods

- default & popular

IPVS mode

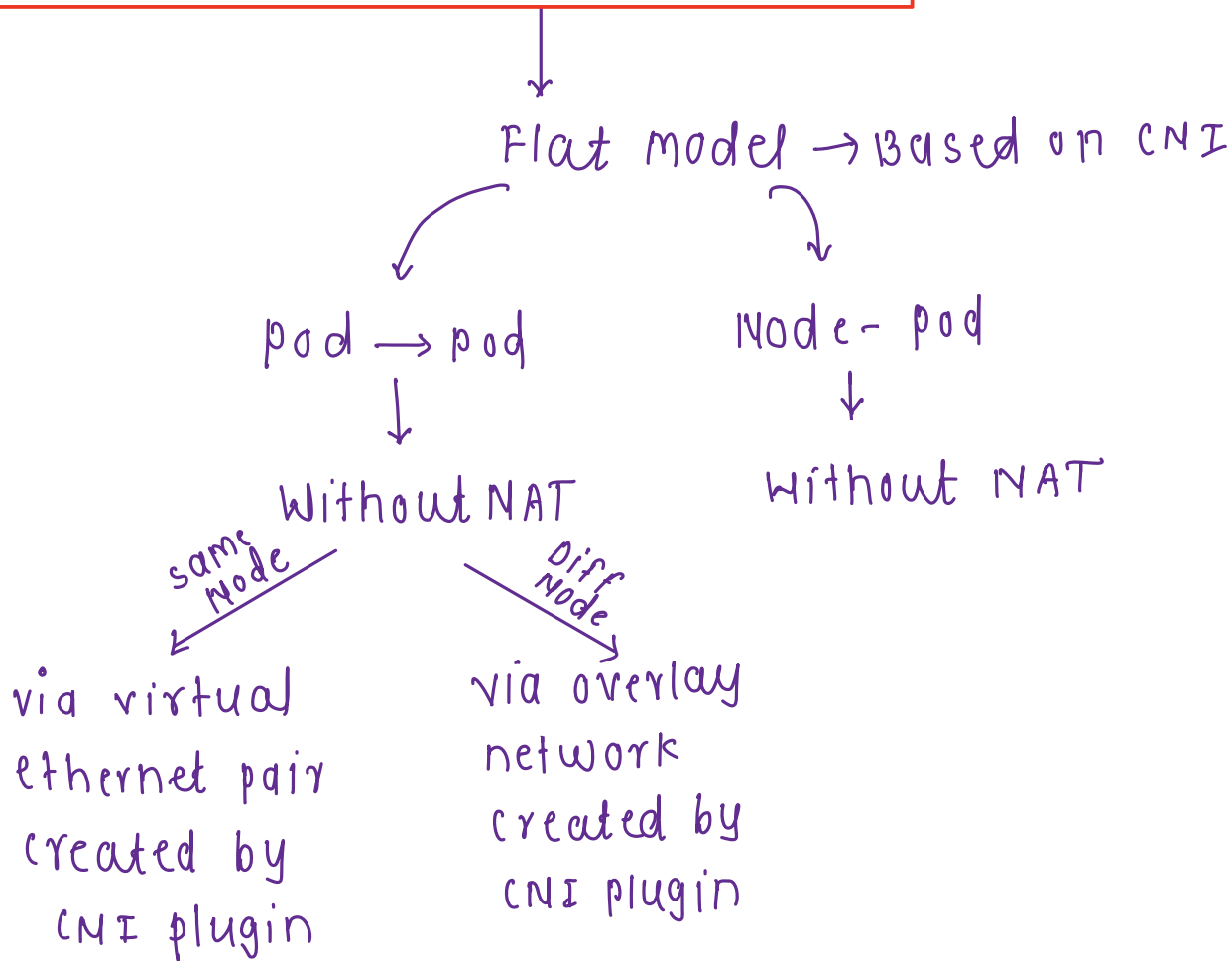
↓
IP virtual server

- mapping rule is inserted into IPVS & not iptables by kube-proxy

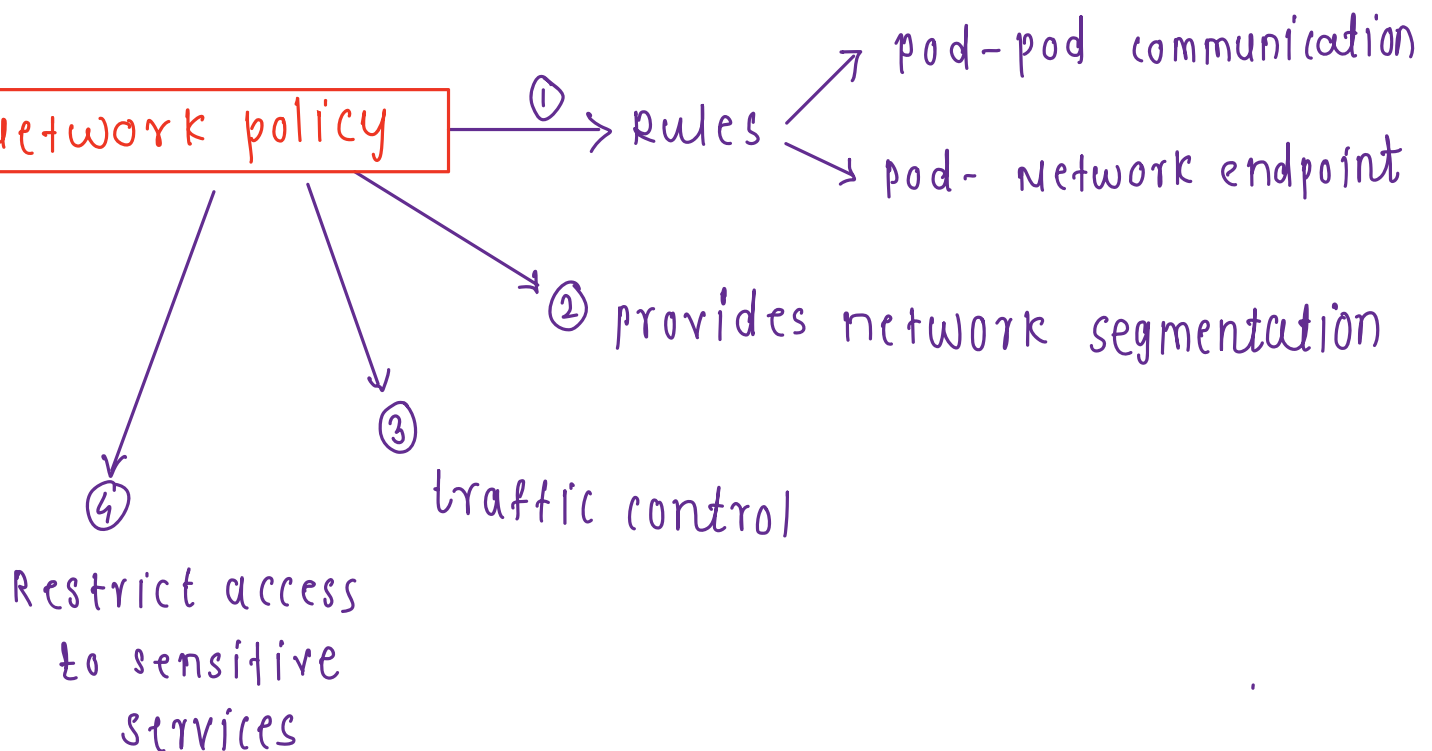
- optimized lookup algorithm for consistent performance

- Adv. Load balancing algorithms
eg. Round robin, Least connection hashing approach

kubernetes networking model



Network policy



kubernetes DNS

↳ [internal DNS] for service discovery

- pod → DNS A record form → pod-ip-addr.namespace-pod.cluster.local
- service → DNS A record → service-name.namespace.svc.cluster.local

challenges of implementing k8s network policies.

- ① Larger clusters $\Rightarrow$ more policy complexity
- ② Policy conflicts $\Rightarrow$ more than 1 policy for same pod
- ③ performance $\downarrow \Rightarrow$ cluster large consumes more resources during policy evaluation leaving few resources.

troubleshooting network connectivity issues in k8s cluster

- ① verify CNI configuration $\rightarrow$ directory $\rightarrow$ /etc/cni/net.d/
- ② check pod label & selector $\rightarrow$ incorrect network policy applied to right pod
- ③ Look for policy conflicts $\rightarrow$ nx policy $\rightarrow$ single pod
- ④ check CNI log files $\rightarrow$ vary $\rightarrow$ /var/log/cni-plug/cni/

Egress Network policy

→ pods are allowed to send traffic outside

Ingress Network policy

→ pods are allowed for incoming traffic

• podselector → select pod matching label

podselectors:

matchLabels:

app: demo

• namespaceSelector → select all pod matching label

namespaceSelector:

matchLabels:

app: demo

• ipBlock → allow traffic from specific cidr range

ipBlock:

cidr: <ip>/24

• from: [] → rules used for ingress policy

↳ default

• to: [] → rules used for egress policy

↳ default

Container Network Interface [CNI]

↳ plugin system that decides Networking in kubernetes.

- Flannel → overlay Network provider
- calico → provides secure network connectivity

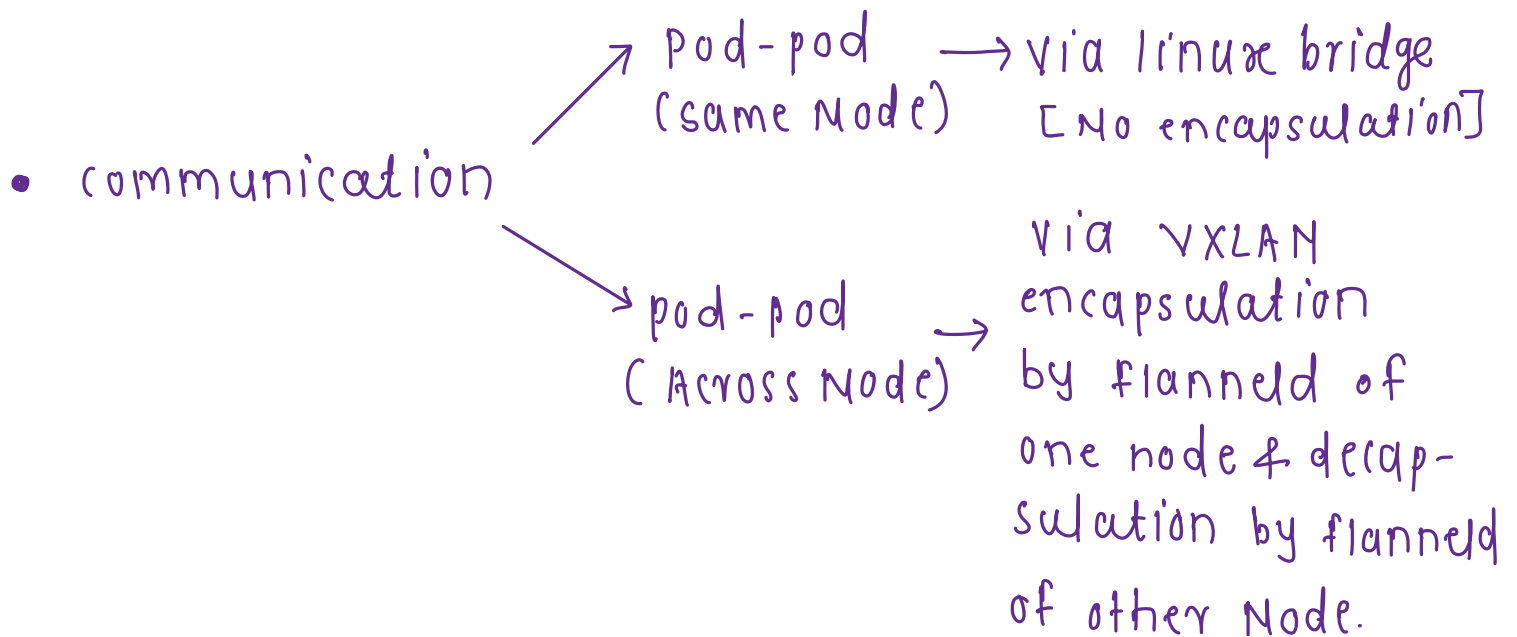
Working of flannel

- Flannel create flat network
- Assign subnet to each node



Each pod get ip from subnet

- Flannel agent maintains routing table mapping subnets to node IP's.



Calico CNI

↳ open source networking & network security solution

• Calico Architecture

- ① **Birds** → BGP routing daemon that distributes routes from Felix to BGP peers [nodes]
- ② **Felix** → manages routes/Acls & other configurations on each host [Node]
- ③ **confd** → Tool that monitors changes in Calico configuration in datastore.
- ④ **IPAM [IP Address mgmt]** → Assign IP's to pods using IP Pool resources
- ⑤ **Typha** → cache between datastore & Felix instance
- ⑥ **Datastore** → stores Calico related configurations

• Working of Calico

- New pod $\xrightarrow{\text{calls}}$ calico → Assign unique IP to pod & add pod to Linux interface
- Each Node has CIDR range → Calico adds route to node's kernel routing table.

- calico allows for network policies for pod communication.
- calico uses pure layer 3 routing → packet transfers from node to node without the encapsulation.

↓
uses BGP [Border Gateway Protocol]

↓
Nodes share routes with each other, so each node knows which node has which pod.

calico Networking Modes

- ① IP IP mode → package encapsulation for inter-node communication.
- ② Direct mode → direct routing without encapsulation
- ③ VXLAN mode → package encapsulation for inter-node communication via VXLAN interface
- ④ pod packet Encryption mode → uses WireGuard tunnels & encrypt & transmit pod traffic between nodes.

service discovery in k8's

- **service registry** → central repository where each service registers network locations/capabilities/metadata
- **service discovery mechanism** → Allows services to dynamically discover & locate other services based on service registry.

• Working

- ① **service registration** → unique DNS Name & IP address is assigned to pod
- ② **DNS resolution** → DNS resolution by other services.
- ③ **Load balancing** → during multiple requests for service, K8 automatically load balances traffic to available pod belong to service.
- ④ **Service updates** → if pods associated with services changes due to scalability → K8 updates service's DNS record for set of available endpoints.

NFS → protocol which allows to mount storage device as local drive on container

- **Benefits**
 - use existing storage volume
 - persistency
 - used to share data between containers
 - simultaneous mounting → NFS can be mounted on multiple nodes → R/W at same time from multiple nodes.

mTLS in kubernetes

↳ Mutual Transport Layer Security.



- zero trust mechanism → Both client & server verify each other using certificates and communicate

